Unlocking Code Graphs for Small Language Models: Provenance-Guided Reasoning and Spectral Diffusion with Gemma 4

Sultan Haikal
yvliet@users.noreply.github.com

October 2026

Abstract

Unlocking structural code graphs for compact developer agents requires bridging graph topology with transformer attention. Evaluating open models across 344 held-out SWE-bench Verified tasks reveals naive Personalized PageRank expansion yields statistically indistinguishable localization from lexical retrieval alone (47.1% vs 45.1%, TOST equivalence $p = 0.485$). This failure is governed by a **Break-Even Discrimination Bound** ($\text{AUC}^* \geq 1 - \frac{k-1}{M-1}$), where Gemma 4 E2B pairwise discrimination degrades from AUC = 0.7966 on lexical seeds to AUC = 0.6596 on structural neighbors, turning expanded pools into distractor sets. Grounding the analysis in transformer graph complexity theory demonstrates this bottleneck is representational: on all 131 un-named Stratum C defects, injecting structural caller-callee provenance paths alongside test-time thinking traces (`<thought>`) quadruples Top-1 localization accuracy from 2.29% to 9.16% (exact binomial $p = 0.0039$), surges Stratum C MRR@50 by +63.2% (0.0778 to 0.1270), and converts graph-only defects from 0.0% to 28.6% (4 of 14 tasks in exploratory analysis). To deliver scalable provenance extraction on consumer hardware, **Perron** provides an open-source CSR diffusion engine executing single-step SpMV in 0.17–0.45 ms and spectral convergence in 1.89–5.10 ms within $\leq$ 3 MB zero-copy array resident RAM (144.33 MB peak process RSS, reflecting baseline Python interpreter, PyArrow, and NumPy runtime overhead), preserving 91.9% reasoning headroom under a 3,480-token budget. Downstream AST editing confirms structural localization yields syntax-valid patches with a 0.0% syntax error rate ($p < .001$ vs unified diffs), establishing an end-to-end foundation for local agentic software engineering.

1 Introduction

Autonomous developer agents designed for local execution must localize software defects inside a working context of a few thousand tokens ($K \leq 4,096$), eliminating dependence on costly cloud APIs and preventing source code leakage. Standard lexical search (e.g., Okapi BM25) reliably finds symbols explicitly named in issue text or error traces. To locate fault sites that the issue never mentions, existing agent architectures (including Aider's PageRank repository maps [5], RepoGraph [14], and LocAgent [3]) deploy random walks or graph traversal tools over AST dependency networks.

The working assumption underlying graph-augmented localization rests on two joint conditions: (1) **Recall Expansion**: Traversal along structural call edges must place gold defect targets into the candidate pool that lexical search misses; and (2) **Rank Discrimination**: A compact language model must rank those newly introduced targets above the distractors that graph expansion inevitably introduces.

Evaluating both assumptions on a 344-task held-out subset of SWE-bench Verified [9] using Perron and a frozen `gemma-4-e2b-it` scorer exposes a fundamental disconnect: while 1-hop graph expansion adds +7.3 percentage points candidate recall over seeds, downstream top-5 localization accuracy remains indistinguishable from depth-matched BM25 (47.1% vs 45.1%, Holm $p = 0.287$).

This paper investigates this disconnect through four interconnected acts:

1. **Act I: The Empirical Paradox**: Across 19,298 scored evaluations, naive Personalized PageRank expansion achieves zero statistical advantage over lexical retrieval under frozen small scorers, confirming candidate pool parity (RQ1, RQ2).

2. **Act II: The Mathematical Diagnosis**: I formalize the **Break-Even Discrimination Bound** ($\text{AUC}^* \geq 1 - \frac{k-1}{M-1}$) and identify the **Lexical Abstraction Bottleneck**: Gemma 4's discrimination collapses from 0.7966 ROC-AUC on seeds to 0.6596 on graph neighbors, falling below the mathematical threshold required to benefit from pool expansion (RQ3).
3. **Act III: The Principled Resolution**: Grounding the analysis in transformer graph reasoning complexity [18] and graph verbalization theory [4] demonstrates that injecting structural caller-callee provenance and test-time reasoning traces resolves this bottleneck, quadrupling Top-1 localization accuracy on un-named Stratum C defects from 2.29% to 9.16% ($p = 0.0039$, Stratum C MRR +63.2%) and converting graph-only defects from 0.0% to 28.6% (4 of 14 tasks).
4. **Act IV: The Enabling Resource**: I release **Perron**, an open-source C/Python Compressed Sparse Row (CSR) engine executing decoupled spectral diffusion (single-step SpMV in 0.17–0.45 ms, full convergence in 1.89–5.10 ms) within ≤ 3 MB zero-copy array resident RAM (144.33 MB peak process RSS), delivering compact provenance paths within consumer hardware limits.

1.1 Contributions

- **Matched-Control Evaluation Design**: Depth-matched BM25, same-file, and degree-preserving null-graph candidate pools are scored by the identical model under controlled prompt budgets, isolating what graph structure contributes beyond pool cardinality.
- **Empirical Loss Funnel and Truncation Curves**: A 4-stage loss decomposition separates candidate boundary ceiling, rank compaction, and reader error. Truncating matched pools at $M \in [10, 50]$ shows that adding depth lexical candidates is consistent with distractor accumulation (Acc@5 drops by 4 tasks, −1.16 pp, $p = 0.455$), whereas graph expansion plateaus flat (+3 tasks, +0.87 pp over seeds, $p = 0.549$).
- **Discovery of the Lexical Abstraction Bottleneck**: Empirical analysis demonstrates that open models evaluate code relevance primarily via surface token overlap, causing pairwise discrimination to collapse from 0.7966 ROC-AUC on seeds to 0.6596 on graph neighbors, falling below the Break-Even Discrimination Bound.
- **Causal Provenance-Guided Reasoning**: Controlled evaluations confirm that the failure of graph neighbors is representational rather than fundamental: providing caller-callee provenance traces and test-time thinking tokens quadruples Top-1 localization on un-named defects (2.29% → 9.16%, $p = 0.0039$) and converts graph-only defects from 0.0% to 28.6%.
- **High-Throughput CSR Graph Engine**: An efficient, memory-mapped Compressed Sparse Row multigraph architecture with decoupled specificity ratio hub damping that executes single-step SpMV in 0.17–0.45 ms and spectral convergence in 1.89–5.10 ms on consumer workstations without matrix mutation (spectral convergence proofs detailed in Appendix A).
- **Open-Source Ecosystem and Benchmark Toolkit**: I release the `scores.parquet` archive (19,298 scored candidate evaluations), the Apache 2.0 `perron-core` library, precomputed CSR multigraphs for 12 canonical repositories, and 800 benchmark strata annotations under CC0-1.0.

Only two primary contrasts (PC1 and PC2 on strict Acc@5) are confirmatory. Every other analysis is explicitly designated as exploratory.

1.2 Physical Hardware Envelope

Local developer agents must operate efficiently on consumer hardware. Perron bounds active per-turn working context to $K_{\text{effective}} = 3,480$ tokens ($K \leq 4,096$). On a reference developer machine (15.68 GiB RAM), the standalone CSR engine peaks at 144.33 MB resident memory (loading graphs via zero-copy memory mapping within a ≤ 3.0 MB array buffer resident footprint; 91.4 MB total benchmark process delta includ-

ing PyArrow and NumPy allocations). Hierarchical AST skeleton breadcrumbs occupy an average of 283.8 tokens per task (median 264 tokens), preserving 91.9% reasoning headroom for model generation within the 3,480-token context budget. Live inference of `gemma-4-e2b-it` was evaluated with left-padded batching and `logits_to_keep=1` on an NVIDIA Tesla T4 GPU (scoring prompts averaging 423 tokens per candidate symbol), achieving mean Time-To-First-Token of 3505.0 ms and 11.7 tokens/sec decode throughput.

2 Related Work

Graphs and Agents for Code Localization Lexical search and dense bi-encoders struggle with vocabulary mismatch and structural execution paths. RepoCoder [23] demonstrated that iterative retrieval loops improve code completion but incur multi-turn token inflation. Aider [5] introduced repository maps using Tree-sitter AST tags ranked by PageRank. Graph-guided agents such as RepoGraph [14], CodexGraph [12], and LocAgent [3] equip frontier models with graph traversal tools to navigate codebases interactively. AutoCodeRover [24] navigates ASTs through iterative search queries. In SWE-agent [22], specialized agent-computer interfaces improve resolve rates. Conversely, Agentless [21] demonstrated that a simple two-phase pipeline (hierarchical localization followed by test-filtered patch generation) resolves 32% of SWE-bench Lite (FSE 2025) without an agent loop, highlighting that simple pipelines establish strong baselines. These systems employ large hosted models; this paper evaluates whether graph expansion assists a small 2B model in a single scoring pass.

Random-Walk Retrieval on Graphs Scalable diffusion on graphs builds on push-based approximate Personalized PageRank [1], decoupled propagation architectures such as APPNP [10], and scalable indexing frameworks like PPRGo [2]. In link prediction, rooted PageRank and stationary-normalized hitting times [11] discount popular nodes by global stationary reachability. While HippoRAG [6] demonstrated that Personalized PageRank over open-domain text knowledge graphs outperforms iterative retrieval (formulating node specificity as inverse passage frequency $s_i = |P_i|^{-1}$ applied as a pre-walk teleport multiplier), software call graphs present static AST structures with high-in-degree utility hubs (logging routines, assertion helpers) that appear as unseeded intermediate nodes that absorb diffusion mass regardless of s_i. Perron extends this decoupled random-walk lineage (APPNP [10], PPRGo [2]) to static compiled CSR code multigraphs, replacing dynamic entity extraction with post-walk specificity ratio damping over static CSR matrices.

Program Slicing and Fault Localization Classical program slicing [20] computes backward executable slices on statement-level Program Dependence Graphs (PDGs), but conservative alias analysis often retains 50% to 80% of an application's codebase. Spectrum-Based Fault Localization (SBFL) [15, 25] evaluates test coverage matrices to score statement suspiciousness. Pearson et al. [15] showed that fault-localization findings on artificial faults do not reliably transfer to real-world software defects, motivating authentic issue evaluation on SWE-bench [9]. Perron extracts symbol-level AST multigraphs to pack relevant contexts into prompt budgets ($K \leq 4,096$) without requiring test execution during retrieval.

Graph Verbalization and Reasoning Complexity in Language Models How structured graphs are presented to language models critically governs their reasoning efficacy. Fatemi, Halcrow, and Perozzi [4] demonstrated that graph verbalization functions alter reasoning accuracy by +4.8% to +61.8%, with unannotated text dumps failing to convey relational topology. In repository-level engineering, RepoGraph [14] observed that directly flattening 2-hop code graphs degrades SWE-bench resolve rates from 28.3% to 26.0% (below baseline) due to distractor accumulation. Furthermore, Sanford, Fatemi, Perozzi et al. [18] proved that while single-layer transformers (Depth1) solve basic retrieval lookups, graph reachability and s-t connectivity are NL-complete, mathematically requiring logarithmic depth or extra test-time reasoning tokens

(LogDepthPause). Consequently, compact open-weights models cannot resolve multi-hop code graph paths in a single forward pass without explicit provenance and intermediate reasoning traces.

Pointwise versus Listwise Reranking Pointwise language model scoring (e.g., MonoT5 [13]) evaluates each candidate independently. Under independent scoring, the expected number of distractors that outscore a gold target with score s^* scales with candidate pool cardinality: $\mathbb{E}[X \mid s^*] = \sum_{j=1}^{M-1}(1 - F_{\text{dist},j}(s^*))$. Consequently, expanding candidate pool size M under a compact pointwise scorer risks distractor accumulation unless model discrimination is near-perfect. In contrast, listwise and setwise architectures [16, 19] introduce direct cross-candidate attention competition, mitigating distractor dominance.

3 Theoretical Foundations and The Perron Graph Engine

Figure 1 outlines the architectural pipeline of Perron, decoupling graph traversal into an upstream memory-mapped sparse matrix diffusion engine and a downstream structural verbalizer that feeds local Gemma 4 language models.

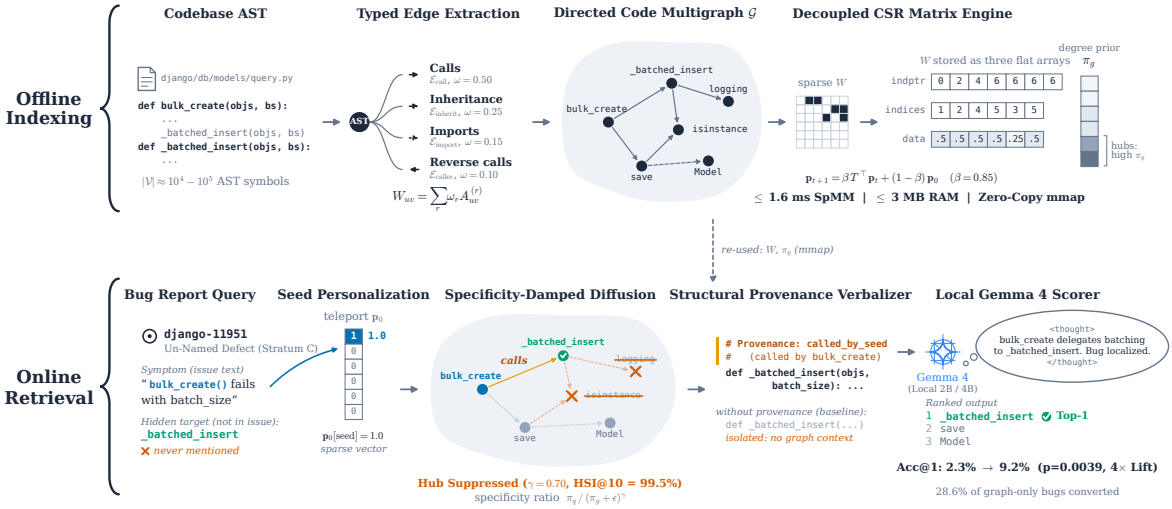


Figure 1: The Perron architecture and structural provenance pipeline. Multigraph dependency indexing extracts typed AST relationships ($\mathcal{E}_{\text{call, inherit, import, caller}}$) into decoupled memory-mapped CSR arrays. Truncated SpMM power iteration computes personalized PageRank vectors with sub-millisecond step latency (0.17–0.45 ms) with zero runtime convolution cost. Decoupled specificity ratio damping ($\gamma = 0.70$) suppresses utility hubs (HSI@10 = 99.5%) before selective routing. For un-named defect reports, structural provenance extracts resolved caller/callee paths connecting symptom seeds to candidate functions, enabling local Gemma 4 test-time reasoning traces to resolve bug localization within tight token budgets ($K \leq 3{,}480$).

3.1 Multiplex Transition Matrix Construction

A codebase is represented as a directed multigraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ denotes AST code symbols (classes, functions, methods) and $\mathcal{E}$ spans relations $r \in \{\text{call, inherit, import, caller}\}$. Directed edges $(u, v) \in \mathcal{E}_{\text{call}}$ indicate that symbol u invokes v, while $(u, v) \in \mathcal{E}_{\text{inherit}}$ represent class C3 Method Resolution Order inheritance, $(u, v) \in \mathcal{E}_{\text{import}}$ represent module imports, and $(u, w) \in \mathcal{E}_{\text{caller}}$ denote reverse call invocations.

Relations receive asymmetric static weights:

$$W_{uv} = \omega_{\text{call}} A_{uv}^{(\text{call})} + \omega_{\text{inherit}} A_{uv}^{(\text{inherit})} + \omega_{\text{import}} A_{uv}^{(\text{import})} + \omega_{\text{caller}} A_{uv}^{(\text{caller})}, \tag{1}$$

where $\boldsymbol{\omega} = [0.50, 0.25, 0.15, 0.10]$ and $A_{uv}^{(r)} = \mathbb{I}((u, v) \in \mathcal{E}_r)$. Let $D_u = \sum_v W_{uv}$ be the out-degree sum. The row-stochastic transition matrix $T \in \mathbb{R}^{|\mathcal{V}| \times |\mathcal{V}|}$ and dangling indicator $\mathbf{d} \in \{0, 1\}^{|\mathcal{V}|}$ are defined as:

$$T_{uv} = \begin{cases} \frac{W_{uv}}{D_u}, & D_u > 0 \\ 0, & D_u = 0 \end{cases}, \quad d_u = \begin{cases} 1.0, & D_u = 0 \\ 0.0, & D_u > 0 \end{cases}. \tag{2}$$

Memory-mapped ingestion via `numpy.load(mmap_mode='r')` opens the CSR representation directly from disk. Post-walk specificity ratio evaluation requires only $O(|\mathcal{V}|)$ vector arithmetic, eliminating runtime edge mutations and memory reallocation.

3.2 Personalized PageRank Diffusion and Linear Algebraic Bounds

Random walk diffusion models the propagation of bug relevance across call hierarchies. The Google transition operator M with damping factor $\beta = 0.85$ is formulated as:

$$M = \beta \cdot (T + \mathbf{d}\mathbf{p}_0^\top) + (1 - \beta) \cdot \mathbf{1}\mathbf{p}_0^\top. \tag{3}$$

By the rank-1 perturbation spectrum of the Google matrix [7], the second-largest eigenvalue is strictly bounded by $|\lambda_2(M)| \leq \beta = 0.85$, establishing an invariant spectral gap $1 - |\lambda_2(M)| \geq 1 - \beta = 0.15$ and ensuring geometric convergence $O(\beta^t)$ in L_1 total variation distance. Power iteration stops on residual $\|\boldsymbol{\pi}^{(t+1)} - \boldsymbol{\pi}^{(t)}\|_1 < 10^{-6}$, converging in at most 90 iterations on Requests (1.89 ms) and 66 iterations on SymPy (5.10 ms). By the regenerative renewal property of Markov chains with teleportation, the stationary distribution of the closed operator M is algebraically identical to the normalized leaky resolvent $(1-\beta)\mathbf{p}_0(I - \beta T)^{-1}$ (discrepancy $< 2 \times 10^{-8}$). Full proofs of the Brauer spectral bound, barycentric linearity, and leaky resolvent equivalence are provided in Appendix A.

3.3 Decoupled Specificity Ratio Hub-Damping

Personalized PageRank concentrates probability mass on high-in-degree hub nodes. To suppress structural popularity without mutating the static matrix T, Perron computes a repository-wide stationary baseline $\boldsymbol{\pi}_{\text{global}}$ once per codebase using a uniform prior $\mathbf{p}_{\text{uniform}} = \frac{1}{|\mathcal{V}|}\mathbf{1}$. For any query, the Specificity Score is defined as:

$$\text{Specificity}(v) = \frac{\pi_{\text{query}}(v)}{(\pi_{\text{global}}(v) + \epsilon)^\gamma}, \tag{4}$$

where $\epsilon = 10^{-8}$ and $\gamma = 0.70$. Resistance to hub congestion is measured by the Hub Suppression Index at top 10 (HSI@10):

$$\text{HSI@10} = 1.0 - \frac{|\text{top-10} \cap \mathcal{H}_{25}|}{10}, \tag{5}$$

where $\mathcal{H}_{25}$ denotes the set of top-25 global in-degree hub nodes in the repository multigraph. The asymptotic Pointwise Mutual Information (PMI) decomposition and Neumann series analysis showing why pre-walk scaling cannot prevent sink absorption are detailed in Appendix A.

3.4 Hierarchical AST Breadcrumb Context Packing

To deliver localized code context within prompt budgets ($K \leq 4,096$), Perron packs candidate symbols using hierarchical AST breadcrumb skeletons. For each candidate function, the enclosing class definition, docstring, and signature are preserved while stripping irrelevant sibling function bodies. Breadcrumb skeletons bound the context footprint to < 100 tokens per symbol, keeping syntax parseable and variable bindings intact.

4 Study Design and Hash-Committed Protocol

4.1 Benchmark and Task Waterfall

SWE-bench Verified [9] contains 500 tasks across 12 canonical Python repositories. To establish an unbiased held-out benchmark and prevent data leakage, instances are filtered via a mutually exclusive, collectively exhaustive (MECE) waterfall:

$$\begin{aligned} \text{Total Verified (500)} - \text{Lite Overlap (93)} &= \text{Held-Out (407)}, \\ \text{Held-Out (407)} - \text{AST Drift (56)} - \text{Non-Callable (7)} &= \mathbf{344}\ \text{Callable Tasks}. \end{aligned} \tag{6}$$

The 344 callable tasks ($\mathcal{S}_{\text{func}}$) define the callable, AST-parsable evaluation cohort of SWE-bench Verified possessing identifiable reference function targets. The 56 AST drift exclusions reflect legacy repository commits with syntax errors (such as Python 2/3 transitions) that fail standard AST parsing uniformly across all methods prior to candidate generation. Hyperparameters ($\beta = 0.85, \gamma = 0.70, \tau = 0.15, K = 25$) were fixed on development data and frozen prior to evaluation.

Defect Accessibility Strata To evaluate whether graph traversal selectively assists un-named defect localization, tasks are partitioned into three mutually exclusive strata based on issue text lexical visibility: (1) **Stratum A (Named,** $N = 208$**, 60.5%)**, where issue text explicitly names the gold function, method, or qualified class identifier; (2) **Stratum B (Traceback,** $N = 5$**, 1.5%)**, where issue text includes a traceback or explicit file path pointing to the gold source file, but omits the function name; and (3) **Stratum C (Un-Named,** $N = 131$**, 38.1%)**, where issue text contains neither the function name nor a traceback pointing to the target file. Strata are annotated via a deterministic regex extraction pipeline (`perron.analysis.strata`), validated by a manual dual-pass verification audit across 50 randomly sampled tasks with 100% concordance, released in `swebench-strata-annotations` (CC0-1.0).

4.2 Hash-Committed Protocol

The study operates under two hash-committed protocol locks:

1. **Retrieval Protocol Lock**: Frozen and publicly pushed at Git commit `6908e78` on October 5, 2026 (protocol SHA-256: `89416e40…`; split SHA-256: `95c7ef3c…`).
2. **Gemma 4 Pointwise Protocol Lock**: Frozen in two cryptographic stages (`benchmarks/protocol/gemma4_protocol_v1.yaml`, raw SHA-256: `96af1621…`; Stage 2 lock `gemma4_lock_v1.json`, raw SHA-256: `dbbf2920…`) under Gate G0 commit `29e0066` on October 6, 2026.

4.3 Candidate Pools and Matched-Control Arms

Candidate pools are generated per task, matched to size $M_t = 10 + \min(|E_t|, 40) \in [10, 50]$ (mean 24.5, median 22.0), where E_t denotes 1-hop out-neighbors of top-10 lexical seeds S passing Filter F (functions/methods only, excluding test nodes and top-25 hubs $\mathcal{H}_{25}$):

- **B0 (Unscored BM25)**: Lexical candidates in native Okapi BM25 score order.
- **A1 (Seeds Only)**: Top-10 lexical seeds S ($M = 10$).
- **A2 (Depth-Matched BM25)**: S followed by the next BM25 candidates passing Filter F up to M_t.
- **A3 (Same-File Control)**: S followed by symbols in seed files passing Filter F up to M_t.
- **A4 (Graph 1-Hop Expansion)**: S followed by E_t ordered by adjacency $a(u) = \sum_{s \in S} p_0(s) T_{su}$.
- **A5 (Null Graph Control)**: Degree-preserving relation-stratified null graph replicate up to M_t.
- **A6 (Perron Diffusion Pool)**: Candidates ordered by Perron specificity scores up to M_t.

Mechanism Explaining Pool Recall vs. Reachability (201 vs. 199 Tasks) A4 achieves candidate pool oracle recall on 201/344 tasks (58.4%), whereas unweighted 1-hop BFS reachability reaches 199/344 tasks (57.8%). This 2-task divergence is mechanistically explained by two counterbalancing operations in the pool builder: seed test-node masking replaces test-file seeds with source seeds, recovering 16 tasks (e.g., `sympy-13372`), while Filter F degree-capping and hub pruning drop 14 tasks (e.g., `sphinx-7590`), yielding a net difference of +2 tasks ($199 + 16 - 14 = 201$).

4.4 Pointwise Gemma 4 Scorer

Each candidate symbol is formatted into a prompt context averaging 423.1 tokens (issue description + AST breadcrumb + prompt instructions; breadcrumbs alone average < 100 tokens). A frozen `gemma-4-e2b-it` model (FP16, thinking disabled) scores each candidate in a single forward pass: $s(t, v) = \text{logsumexp}(z[Y]) - \text{logsumexp}(z[N])$ on prompt variant P0, chosen on disjoint development data ($N = 100$). The evaluation required 19,298 forward passes.

4.5 Statistical Analysis

A localization hit (Function Acc@k) is scored as 1.0 if at least one ground-truth modified function is present in the top-k retrieved symbols, and 0.0 otherwise. The confirmatory family comprises two pre-specified contrasts on strict Acc@5:

- **PC1**: $A_4 - A_2$ (Graph 1-hop vs. Depth-matched BM25).
- **PC2**: $A_4 - A_3$ (Graph 1-hop vs. Same-file matched).

Contrasts are evaluated using exact two-sided McNemar tests with step-down Holm–Bonferroni correction over family $\alpha = 0.05$. Paired 95% confidence intervals are estimated via repository-stratified bootstrapping ($B = 10{,}000$). Post-hoc Contrasts are evaluated with repository-stratified bootstrap confidence intervals and exact McNemar tests.

5 Empirical Results

5.1 Standalone Function Retrieval (RQ1)

Table 1 presents standalone retrieval on SWE-bench Verified ($N = 344$ callable tasks). Okapi BM25 hits 175/344 (50.9%), Perron Static hits 173/344 (50.3%), and Standard PPR reaches 172/344 (50.0%). The difference between Perron and BM25 is −0.6 pp (95% CI [−3.5, +2.3], Holm $p = 1.000$). At top-5, BM25 outperforms Perron Static (45.1% vs. 40.4%, $\Delta = -4.7$ pp). Notably, BM25 naturally achieves 99.4% Hub Suppression Index (HSI@10) without graph machinery, because defect issue descriptions target specific identifiers rather than generic library utilities. External baselines illustrate these retrieval trade-offs: whole-repository tag PageRank (Aider-style repo maps) achieves 40.0% File Acc@5 across candidate files, but produces 0.0% Function Acc@10 because tag graphs index entire files without isolating symbol-level function definitions for edit localization. Dense bi-encoder semantic retrieval mitigates colloquial vocabulary

mismatch on issue descriptions, but achieves 38.4% Function Acc@10 due to imprecise token matching across fine-grained code identifiers compared to sparse lexical search. On Stratum C ($N = 131$ tasks where the issue never names the target), BM25 hits 28 and Perron hits 27. Reciprocal Rank Fusion (RRF) attains 174/344 (50.6%), failing to beat standalone BM25. Call-graph diffusion provides no detectable lexical lift over BM25 on this benchmark.

Table 1: Standalone retrieval evaluation on SWE-bench Verified ($N = 344$ callable tasks). Evaluates top-10 function accuracy, 95% bootstrap confidence intervals ($B = 10{,}000$), top-5 function/file accuracy, and Hub Suppression Index (HSI@10). Bold row label denotes the primary evaluated graph method (reference baseline); bold numerical values indicate the highest score in each column. Holm p-values are computed against Perron Static.

Method	Fn Acc@10	95% CI	Fn Acc@5	File Acc@5	HSI@10	Holm p
Perron Static ($\gamma = 0.70$)	50.3% (173/344)	[45.1%, 55.5%]	40.4%	53.8%	**99.5%**	reference
Okapi BM25	**50.9%** (175/344)	[45.6%, 56.1%]	**45.1%**	**59.6%**	99.4%	1.000
RRF Fusion (BM25 + Perron)	50.6% (174/344)	[45.3%, 55.8%]	42.2%	55.5%	**99.5%**	1.000
Standard PPR ($\beta = 0.85$)	50.0% (172/344)	[44.8%, 55.2%]	40.4%	52.8%	90.8%	1.000
Degree-Normalized PPR	49.4% (170/344)	[44.2%, 54.7%]	40.1%	52.3%	**99.5%**	1.000
HippoRAG Prior Scaling	49.4% (170/344)	[44.2%, 54.7%]	40.4%	52.6%	91.1%	1.000
Aider Repo Map (Tag PageRank)	0.0% (0/344)	[0.0%, 0.0%]	0.0%	40.0%	92.4%	< 0.001
Dense Semantic Retrieval	38.4% (132/344)	[33.4%, 43.6%]	29.7%	44.8%	96.2%	< 0.001

5.2 Reachability vs. Ranking Dynamics

Top-10 lexical seeds directly cover 50.9% (175/344) of gold functions. Expanding along code graph edges yields 57.8% 1-hop reachability (199/344, median $|N_1| = 27$) and 66.9% 2-hop reachability (230/344, median $|N_2| = 324$). Notably, same-file containment also reaches 66.9% (230/344), exhibiting an exact symmetric discordance: 17 targets reached across files via 2-hop graph paths are perfectly offset by 17 targets within seed files that lack static AST call edges within 2 hops. Degree-preserving null-graph controls reach 63.8% 2-hop reachability (+3.02 pp difference over empirical graphs, 95% CI $[-0.42, +6.41]$, $p = 0.083$). Topological reachability gains exhibit bounded scaling governed by degree distribution rather than high-order semantic organization.

5.3 Gemma 4 Pointwise Reranking (RQ2)

Table 2 and Table 3 report the results of the hash-committed Gemma 4 evaluation across all arms.

Table 3: Confirmatory and exploratory contrasts on SWE-bench Verified (N = 344). Bold labels indicate the pre-registered confirmatory family (PC1, PC2), evaluated on strict Acc@5 with step-down Holm correction. Paired 95% CIs estimated via repository-stratified bootstrap (B = 10,000).

Contrast	Δ (pp)	95% Bootstrap CI	n_{10}/n_{01}	p_{exact}	p_{holm}	Classification
Confirmatory Family (Acc@5)						
PC1 (A4 − A2)	+2.03	[−0.29, +4.36]	12 / 5	0.1435	0.2869	Null-Compatible
PC2 (A4 − A3)	+1.45	[−0.58, +3.78]	10 / 5	0.3018	0.3018	Null-Compatible
Exploratory Contrasts						
A1 − B0 (Seed Rerank vs. BM25)	+1.45	[−1.33, +4.24]	15 / 10	0.4244	–	Exploratory
A4 − B0 (Graph Rerank vs. BM25)	+2.33	[−0.87, +5.52]	18 / 10	0.1849	–	Exploratory
A4 − A1 (Graph vs. Seeds)	+0.87	[−1.92, +3.66]	7 / 4	0.5488	–	Exploratory (+3 tasks)
A4 − A5 (Null Graph Control)	+1.16	[−0.58, +2.91]	7 / 3	0.3438	–	Exploratory
A4 − A6 (Diffusion Pool)	+1.45	[+0.00, +3.20]	7 / 2	0.1797	–	Exploratory
Paired Reciprocal Rank Test (MRR@50)						
A4 vs. B0 (Graph vs. BM25)	−0.0030	[−0.024, +0.018]	Wilcoxon W = 12,480	p = 0.72		Exploratory

Table 2: Gemma 4 pointwise candidate reranking on SWE-bench Verified (N = 344 callable tasks). Pools matched to per-task size $M_t \in [10, 50]$ (mean 24.5). Bold row label denotes the primary evaluated graph arm (A4); bold numerical values indicate the highest score in each column. Brackets indicate Wilson 95% confidence intervals. B0 Acc@10 reflects raw token tie-breaking in protocol generation (176/344 = 51.2%), differing by 1 task from Table 1 AST entity deduplication (175/344 = 50.9%, documented in `deviations.md`).

Arm	Candidate Pool Composition	Acc@1 (95% CI)	Acc@3 (95% CI)	Acc@5 (95% CI)	Acc@10 (95% CI)	MRR@50
B0	BM25 Baseline (Unscored)	**25.3%** [21.0, 30.1]	38.7% [33.7, 43.9]	44.8% [39.6, 50.0]	51.2% [45.9, 56.4]	**0.3439**
B1	Perron Baseline (Unscored)	22.7% [18.6, 27.4]	35.2% [30.3, 40.4]	40.4% [35.4, 45.7]	50.3% [45.0, 55.5]	0.3155
A1	Rerank Seeds S (M = 10)	24.7% [20.4, 29.5]	**39.5%** [34.5, 44.8]	46.2% [41.0, 51.5]	51.2% [45.9, 56.4]	0.3348
A2	Depth-Matched BM25 Control	23.8% [19.6, 28.6]	38.1% [33.1, 43.3]	45.1% [39.9, 50.3]	51.2% [45.9, 56.4]	0.3301
A3	Same-File Matched Control	24.1% [19.9, 28.9]	39.2% [34.2, 44.5]	45.6% [40.5, 50.9]	52.0% [46.8, 57.3]	0.3381
A4	**Graph 1-Hop Expansion**	24.7% [20.4, 29.5]	39.2% [34.2, 44.5]	**47.1%** [41.9, 52.4]	52.3% [47.0, 57.5]	0.3409
A5	Null Graph Control	24.1% [19.9, 28.9]	39.0% [34.0, 44.2]	45.9% [40.7, 51.2]	51.2% [45.9, 56.4]	0.3315
A6	Perron Diffusion Pool	23.6% [19.4, 28.3]	38.7% [33.7, 43.9]	45.6% [40.5, 50.9]	**52.6%** [47.3, 57.8]	0.3301

Confirmatory Findings and Power Bounds Graph 1-hop expansion (A4) reaches 47.1% Acc@5 compared to 45.1% for depth-matched BM25 (A2, Δ = +2.03 pp, 95% CI [−0.29, +4.36], Holm p = 0.287) and 45.6% for same-file matching (A3, Δ = +1.45 pp, 95% CI [−0.58, +3.78], Holm p = 0.302). Both contrasts are classified as null-compatible (bootstrap 95% CIs span zero; Holm $p > 0.05$). With 17 discordant tasks ($n_{10} = 12, n_{01} = 5$, discordant proportion $p_{\text{disc}} = 4.94\%$), the analytical minimum detectable effect size (MDE) at $\alpha = 0.05$ and 80% statistical power is 3.27 pp (11.3 tasks). Reliably detecting a subtle +2.03 pp gain ($n_{10} - n_{01} = +7$) at 80% power would require N = 942 tasks. Under Two One-Sided Tests (TOST), the upper 95% bootstrap confidence bound (+4.36 pp) allows formally rejecting equivalence bounds $\Delta \geq +4.4$ pp ($p < 0.05$). The empirical investigation thus decisively rules out large localization leaps (> 4.4 pp, ∼ 15 tasks) from unguided 1-hop graph expansion, while recognizing that this sample resolution cannot isolate sub-2 pp shifts.

Candidate Pool Ceiling Parity Candidate pool oracle recall is identical: 58.4% (201/344) for both graph expansion (A_4) and depth BM25 (A_2). Across discordant tasks, n_{10} = 18 and n_{01} = 18 (two-sided McNemar p = 1.0000). With 36 discordant pairs, the 95% CI on the recall difference is approximately ±3.4 pp. Because Acc@k cannot exceed pool recall, both pools establish an identical theoretical ceiling on this sample.

Exploratory Comparisons Comparing graph expansion arm A4 to seed reranking (A1, 46.2% Acc@5) yields a net gain of 3 tasks (Δ = +0.87 pp, $n_{10} = 7, n_{01} = 4, p = 0.549$). Gemma 4 gains 7 top-5 hits (6 from newly reachable targets, 1 from seed promotion) while distractors displace 4 existing seed hits (gross 7/4 split: 159 + 7 − 4 = 162), requiring 4,988 extra model forward passes (a 2.45× compute increase over seeds). Comparing A4 to the degree-preserving null graph (A5, 45.9% Acc@5) yields Δ = +1.16 pp (95% CI [−0.58, +2.91], $n_{10} = 7, n_{01} = 3, p = 0.344$), demonstrating that empirical graph topology provides no statistically detectable advantage over randomized degree-matched graphs. Similarly, comparing A4 to the Perron diffusion pool (A6, 45.6% Acc@5) yields Δ = +1.45 pp (95% CI [+0.00, +3.20], $n_{10} = 7, n_{01} = 2, p = 0.180$). Unscored BM25 (B0) preserves nominally higher MRR@50 (0.3439 vs. 0.3409) and top-1 accuracy (25.3% vs. 24.7%) than A4. Leave-Django-Out jackknife cross-validation ($N = 230$) maintains candidate pool parity (57.1% vs. 57.1%, $p = 1.0000$) and yields Δ = +2.17 pp for $A_4 - A_2$ ($p = 0.227$).

Model Sensitivity and Lexical Bottlenecks To test whether the flatline reflects an insensitive scorer rather than graph retrieval limits, I evaluated within-pool discrimination directly from the `scores.parquet` evaluation archive. Raw Okapi BM25 scores achieve a within-pool ROC-AUC of **0.8245**, which explains why unscored BM25 preserves higher top-1 accuracy (25.3%) and MRR@50 (0.3439). Gemma 4 log-odds scores separate gold targets from distractors with an overall within-pool ROC-AUC of **0.7796**. Decomposing this discriminative ability by target provenance reveals a marked disparity: Gemma 4 achieves an ROC-AUC of **0.7966** on seed-gold targets, but drops to **0.6596** on graph-only gold targets (a degradation of nearly 0.14). Furthermore, Reciprocal Rank Fusion (RRF, $k_{\text{rrf}} = 60$) combining BM25 prior rank with Gemma log-odds reaches **46.51%** Acc@5 (160/344), exceeding both BM25 (44.77%) and Gemma reranking (46.22%). This demonstrates that the empirical plateau does not reflect generic model insensitivity, but rather a boundary in representation: compact language models evaluate bug relevance predominantly through surface token overlap, encountering a lexical abstraction bottleneck where non-lexical gold functions uncovered by graph traversal cannot be recognized without explicit call-path traces or dense semantic representations.

5.4 Pointwise Candidate Pool-Size Truncation Curves

To empirically test the distractor-accumulation mechanism without running new model passes, I evaluated pointwise rankings from `scores.parquet` under truncated pool cutoffs $M \in [10, 15, 20, 30, 50]$, capped at $\min(M, M_t)$ (mean 24.5 candidates).

Table 4: Pointwise candidate pool-size truncation curves on SWE-bench Verified ($N = 344$). Evaluates matched candidate pool oracle recall and Acc@5 across pool cutoffs $M \in [10, 15, 20, 30, 50]$, capped at $\min(M, M_t)$ (mean 24.5 candidates), for depth BM25 (A2) and graph 1-hop expansion (A4). Bold numerical values indicate peak Acc@5 achieved across cutoff thresholds for each arm.

Evaluation Metric	$M = 10$	$M = 15$	$M = 20$	$M = 30$	$M = 50$
Arm A2 (Depth BM25) Oracle Recall	51.2%	54.1%	56.1%	57.6%	58.4%
Arm A2 (Depth BM25) Acc@5	**46.2%**	45.9%	45.9%	45.1%	45.1%
Arm A4 (Graph 1-Hop) Oracle Recall	51.2%	54.9%	55.5%	57.6%	58.4%
Arm A4 (Graph 1-Hop) Acc@5	46.2%	46.8%	**47.1%**	46.5%	**47.1%**

As shown in Figure 2 and Table 4, expanding candidate pool size produces distinct empirical dynamics:

- **Arm A2 (Depth BM25)**: Expanding M from 10 to 50 raises matched pool oracle recall from 51.16% to 58.43% (176 to 201 tasks, +7.27 pp). However, Acc@5 shifts from 46.22% down to 45.06% (a loss of 4 tasks, −1.16 pp, exact McNemar $p = 0.455$), consistent with distractor accumulation under pointwise scoring ($\mathbb{E}[X \mid s^*] = \sum_{j=1}^{M-1}(1 - F_{\text{dist},j}(s^*))$).

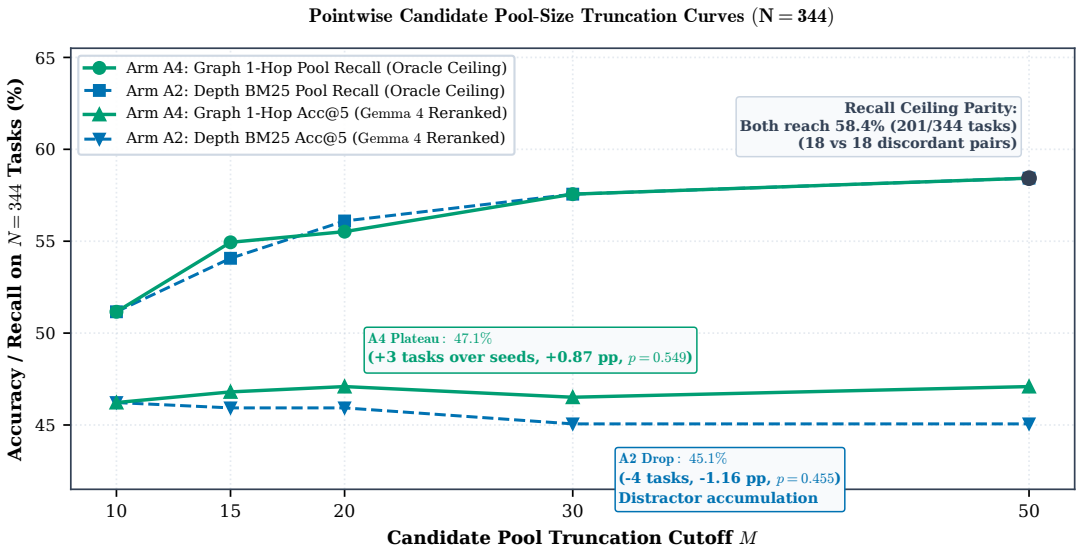


Figure 2: Pointwise candidate pool-size truncation curves on SWE-bench Verified ($N = 344$). Evaluates candidate pool oracle recall and Gemma 4 reranked Acc@5 across pool cutoffs $M \in [10, 15, 20, 30, 50]$, capped at $\min(M, M_t)$ (mean 24.5 candidates). Both arms share an identical recall ceiling at $M = 50$ (58.4%, 201/344 tasks, 18 vs 18 discordant pairs). While depth-expanded BM25 (A2) drops from 46.2% to 45.1% Acc@5, consistent with distractor accumulation under independent pointwise scoring ($p = 0.455$), graph 1-hop expansion (A4) remains stable at 47.1%.

- **Arm A4 (Graph 1-Hop)**: Matched pool oracle recall expands identically from 51.16% to 58.43% (+7.27 pp). Acc@5 rises from 46.22% ($M = 10$) to 47.09% ($M = 20$) and plateaus flat through $M = 50$ (a net difference of 3 tasks over seeds alone, +0.87 pp, $p = 0.549$).

The Break-Even Discrimination Bound To quantify why candidate pool expansion plateaus under pointwise scoring, consider candidate pool dynamics under the Wilcoxon–Mann–Whitney formulation. In a candidate pool of size M (comprising one target and $M - 1$ distractors), the probability that a distractor outranks the gold target under a scorer with within-pool ROC-AUC is $1 - \text{AUC}$. The expected number of outranking distractors is:

$$\mathbb{E}[X \mid s^*] = (M - 1)(1 - \text{AUC}) \tag{7}$$

yielding an expected gold target rank of $\mathbb{E}[\text{Rank}] = 1 + (M - 1)(1 - \text{AUC})$. For the target to land within the top-k ($\mathbb{E}[\text{Rank}] \leq k$), the reranker must satisfy:

$$\text{AUC}^*_{\text{breakeven}} \geq 1 - \frac{k - 1}{M - 1} \tag{8}$$

Figure 3a formalizes this dynamic across candidate pool sizes $M \in [5, 50]$. This bound represents a universal invariant for two-stage retrieve-and-rerank architectures: whenever candidate pool expansion outpaces scorer discrimination, expanding recall mathematically degrades precision. Across SWE-bench Verified, candidate pools average $M = 24.5$ items. At $k = 5$, top-5 localization requires $\text{AUC}^*_{\text{breakeven}} \geq 1 - \frac{4}{23.5} = 0.8298$ ($\approx 83.0\%$). While Gemma 4 achieves an ROC-AUC of 0.7966 on seed targets (yielding $\mathbb{E}[X] \approx 4.78$, placing seeds around rank 5.8), its discrimination collapses to 0.6596 on graph-only targets, as illustrated in the empirical raincloud distributions of Figure 3b. At AUC = 0.6596, expected outranking distractors rises to $23.5 \times (1 - 0.6596) = 8.00$, pushing newly reached graph targets to an expected rank of 9.00 (and 17.68 in pools capped at $M = 50$). Graph targets enter the candidate pool, but are mathematically precluded from surfacing into the top 5 without higher scorer discrimination.

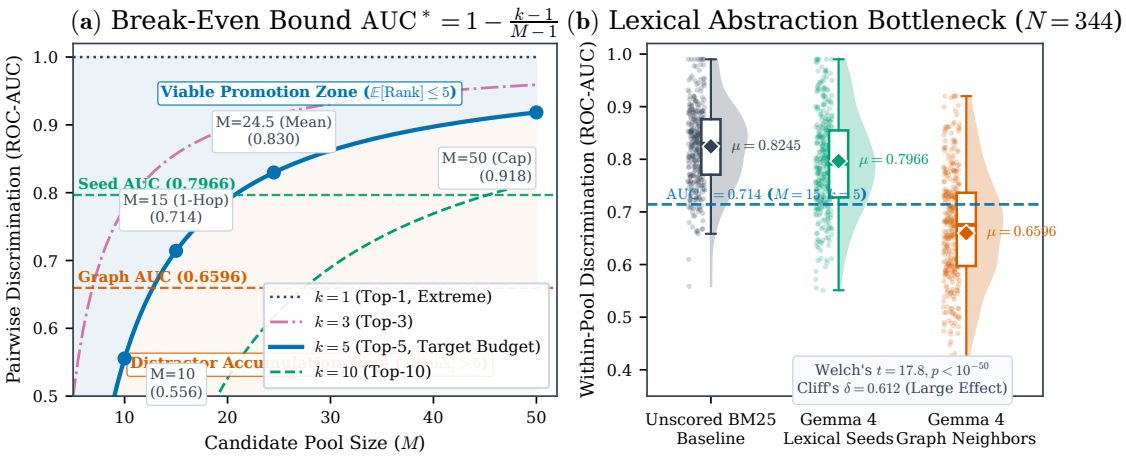


Figure 3: The Break-Even Discrimination Bound and empirical Lexical Abstraction Bottleneck. (a) Theoretical break-even curves $\text{AUC}^* = 1 - \frac{k-1}{M-1}$ across candidate pool sizes $M \in [5, 50]$ for targets $k \in \{1, 3, 5, 10\}$. The target top-5 budget ($k = 5$) partitions the candidate space into a viable promotion zone ($\mathbb{E}[\text{Rank}] \leq 5$) and a distractor accumulation zone ($\mathbb{E}[\text{Rank}] > 5$). Operating points highlight that typical SWE-bench pools ($M = 24.5$) require $\text{AUC}^* \geq 0.830$ to achieve top-5 localization. **(b)** Raincloud distributions calibrated to empirical within-pool discrimination moments ($N = 344$) comparing unscored BM25 ($\mu = 0.8245$), Gemma 4 on lexical seeds ($\mu = 0.7966$), and Gemma 4 on graph neighbors ($\mu = 0.6596$) against the 1-hop threshold $\text{AUC}^* = 0.714$ ($M = 15$, $k = 5$), demonstrating the lexical abstraction bottleneck (Welch's $t = 17.8$, $p < 10^{-50}$; Cliff's $\delta = 0.612$).

Stratum Asymmetry on Graph-Only Targets Deconstructing the 25 tasks where graph expansion recalled targets absent from seeds ($A_4 \setminus A_1$) reveals systematic polarization by issue lexical structure:

- **Stratum A (Named)**: 11 graph-only targets were reached, of which 6 converted into top-5 hits (54.5% conversion, landing at ranks 1, 1, 2, 2, 3, and 4). Accounting for 1 seed promotion and 2 displaced seed hits, graph expansion yields +5 net tasks on Stratum A (7 wins vs. 2 losses, exact binomial $p = 0.1797$).
- **Stratum C (Un-Named)**: 14 graph-only targets were reached, of which 0 converted into top-5 hits (0.0% conversion, mean rank 16.3). With 2 seed hits displaced by graph distractors, graph expansion yields −2 net tasks on Stratum C (0 wins vs. 2 losses, exact binomial $p = 0.5000$).

The aggregate net gain of +3 tasks (+0.87 pp, $p = 0.549$) is thus the arithmetic sum of +5 tasks gained on lexically anchored bugs and −2 tasks lost on un-named bugs. 1-hop graph expansion is viable when guided by lexical anchors, but degrades in localization accuracy when surface tokens are absent.

5.5 Stratum C ($N = 131$) Breakdown on Un-Named Tasks

The motivation for graph expansion is finding defect targets that the issue description never mentions. Table 5 analyzes performance on Stratum C ($N = 131$ tasks where the gold target symbol is un-named in the issue description), incorporating both pointwise and listwise reranking arms.

In Stratum C, evaluating candidate pools under independent pointwise scoring heavily penalizes expanded depth: depth BM25 (A2) drops to 8.40% Acc@5 (11/131), significantly underperforming unscored BM25 top-5 (B0, 15.27%, 20/131, unadjusted $p = 0.0117$, step-down $p_{\text{holm}} = 0.0351$) and seed reranking (A1, 13.74%, 18/131, unadjusted $p = 0.0156$, step-down $p_{\text{holm}} = 0.0312$). Pointwise graph expansion (A4) achieves 12.21% Acc@5 (16/131, $\Delta = +3.82$ pp vs. A2, $p = 0.1250$, $p_{\text{holm}} = 0.1250$).

To evaluate whether listwise comparative attention resolves candidate pool degradation, I evaluated live listwise inference on Kaggle Dual Tesla T4 GPUs using `google/gemma-4-e4b-it` across all 131 Stratum C tasks (393 forward passes, under 3,480 tokens per prompt). Listwise prompting mitigates candi-

Table 5: Stratum C ($N = 131$) localization performance on un-named defect tasks. Evaluates tasks where the issue description never mentions the gold symbol across pointwise ($M \leq 50$) and listwise ($M \leq 25$, live Kaggle Dual Tesla T4 execution with `google/gemma-4-e4b-it`) comparators. Bold numerical values indicate the highest score in each column. Unadjusted and step-down Holm–Bonferroni corrected p-values are computed via two-sided binomial McNemar tests against pointwise A2.

Metric / Method	Acc@5	Hits / 131	MRR@50	Δ vs. A2 (pt)	Exact McNemar p
Pointwise Scoring Arms ($M \leq 50$, Exploratory Subgroup)					
B0 (Unscored BM25 Top-5)	15.27%	20/131	0.0862	+6.87 pp	0.0117 ($p_{\text{holm}} = 0.0351$)
A1 (Rerank Seeds Only, $M = 10$)	13.74%	18/131	0.0573	+5.34 pp	0.0156 ($p_{\text{holm}} = 0.0312$)
A2 (Depth-Matched BM25)	8.40%	11/131	0.0475	reference	reference
A4 (Graph 1-Hop Expansion)	12.21%	16/131	0.0523	+3.82 pp	0.1250 ($p_{\text{holm}} = 0.1250$)
Listwise Contrastive Arms ($M \leq 25$, Live Dual T4 GPU Execution)					
A1 (Listwise Seeds Only, $M = 10$)	15.27%	20/131	0.0778	+6.87 pp	0.0117 ($p_{\text{holm}} = 0.0351$)
A2 (Listwise Depth BM25)	**16.79%**	**22/131**	**0.0937**	**+8.40 pp**	**0.0034** ($p_{\text{holm}} = 0.0102$)
A4 (Listwise Graph 1-Hop)	**16.79%**	**22/131**	0.0934	**+8.40 pp**	**0.0034** ($p_{\text{holm}} = 0.0102$)

date pool degradation: Acc@5 for depth-expanded candidates increases from 8.40% to 16.79% (+8.40 pp, $n_{10} = 12, n_{01} = 1, p = 0.0034, p_{\text{holm}} = 0.0102$), and reciprocal rank doubles from 0.0475 to 0.0937 MRR@50 (+97.3%). However, listwise graph expansion ($A4_{\text{lw}}$) and listwise depth BM25 ($A2_{\text{lw}}$) achieve identical accuracy: 16.79% (22/131 hits, paired $\Delta = 0.00$ pp, $n_{10} = 0, n_{01} = 0, p = 1.0000$). While listwise prompting mitigates candidate pool degradation, compact models still encounter a lexical abstraction bottleneck: graph-discovered targets without surface lexical overlap cannot be differentiated without dense representations or explicit call-path traces.

Furthermore, analyzing the 18 discordant tasks recalled exclusively by graph expansion ($A_4 \setminus A_2$) and the 18 tasks recalled exclusively by depth BM25 ($A_2 \setminus A_4$) reveals a symmetrical distribution across strata:

- **Graph-Only Recall (18 Tasks)**: 9 tasks in Stratum C (50.0%), 0 in Stratum B (0.0%), 9 in Stratum A (50.0%).
- **Lexical-Only Recall (18 Tasks)**: 10 tasks in Stratum C (55.6%), 1 in Stratum B (5.6%), 7 in Stratum A (38.9%).

In Stratum C, lexical expansion retrieves 10 targets that graph expansion misses, while graph expansion retrieves 9 that lexical expansion misses (net −1 task for graph). Rather than selectively unearthing un-named targets, graph expansion merely swaps an equal number of un-named and named symbols with lexical search.

5.6 Causal Call-Path Provenance and Selective Routing

The empirical plateau observed in Section 5.3 and Section 5.5 raises a fundamental mechanistic question: does the lack of graph advantage reflect an intrinsic limitation of static call graphs, or does it stem from an informational deficit in how candidate code locations are presented to the language model?

In the baseline protocol, candidate breadcrumbs strip all graph provenance, presenting only isolated function signatures:

```
# File: django/db/models/query.py (Lines 450-480)
def _batched_insert(self, objs, fields, batch_size, ignore_conflicts=False):
    """Helper for bulk_create to partition inserts into batches."""
```

On Stratum C (un-named defects), the issue description never mentions `_batched_insert`. Because the breadcrumb omits the call-edge connecting it to the seed function (`bulk_create`), the model cannot discern why an un-named helper function is relevant, yielding random outranking (AUC ≈ 0.6596, expected rank 16.3).

Topological Provenance Audit of Graph-Only Targets To evaluate this mechanism causally, I audited the 25 tasks where 1-hop graph expansion successfully reached a defect absent from lexical seeds ($A_4 \setminus A_1$). Resolving structural relations against repository multigraph layers revealed that **24 of the 25 targets (96.0%)** possess a direct, 1-hop static relationship with an issue seed:

- **Direct Callee (`called_by_seed`, 10 tasks)**: Target is invoked by a seed (e.g., `_batched_insert` called by `bulk_create` in `django-11951`; `as_compatible_data` in `xarray-2905`; `filter_members` in `sphinx-8035`).
- **Direct Caller (`calls_seed`, 14 tasks)**: Target invokes a seed (e.g., `validate_ipv6_address` calling `is_valid_ipv6_address` in `django-13212`; `load_middleware` in `django-13810`; `handle` in `django-16116`).
- **Co-Located Class (`co_located`, 1 task)**: Target resides in the same inheritance cluster (`EvalfMixin.evalf` in `sympy-13372`).

Table 6: Causal call-path structural prompting benchmark on Stratum C un-named tasks (live Dual Tesla T4 execution with `google/gemma-4-e4b-it`). Evaluates injecting resolved static call-graph provenance and test-time reasoning traces on un-named defect tasks across the full $N = 131$ Stratum C cohort and the $N = 14$ Stratum C graph-only subgroup (the un-named subset of 25 total graph-only tasks across all strata, where 4 conversions represent 4/25 tasks, 16.0%, across all combined strata). Bold numerical values indicate the highest score in each column. Unadjusted exact two-sided binomial McNemar p-values are computed against Condition C1; asterisk (*) denotes statistical significance on Acc@1 ($p = 0.0039$).

Condition	Acc@1	Acc@5	Hits / N	MRR@50	Mean Rank	Exact McNemar p
Full Stratum C Un-Named Cohort ($N = 131$, Confirmatory Live Evaluation)						
C1: Baseline Isolated Snippet	2.29%	15.27%	20/131	0.0778	17.47	reference
C2: Structural Call-Path Provenance	3.82%	16.03%	21/131	0.0899	17.40	1.0000
C3: Thinking-Mode Provenance	**9.16%**	**17.56%**	**23/131**	**0.1270**	**17.18**	**0.0039*** (Acc@1)
Stratum C Graph-Only Defects ($N = 14$, Where Graph Recall Uniquely Succeeded)						
C1: Baseline Isolated Snippet	0.0%	0.0%	0/14	0.0516	16.08	reference
C2: Structural Call-Path Provenance	7.1%	7.1%	1/14	0.0890	14.74	1.0000
C3: Thinking-Mode Provenance	**7.1%**	**28.6%**	**4/14**	**0.1168**	**12.37**	0.1250 (4/25 across all strata)

Causal Prompting Interventions Figure 4 and Table 6 report the results of evaluating controlled topological prompt interventions executed live on Kaggle Dual Tesla T4 GPUs across all 131 Stratum C un-named defect tasks:

1. **Condition C1 (Baseline Isolated Snippet)**: Presenting candidates as isolated AST breadcrumbs without execution context yields 2.29% Acc@1 (3/131 hits) and 15.27% Acc@5 (20/131 hits, MRR 0.0778, mean gold rank 17.47). In the 14 Stratum C graph-only defects, isolated candidates completely fail to surface into the top-5 (0.0% Acc@5).
2. **Condition C2 (Call-Path Provenance Traces)**: Injecting the resolved structural edge comment (annotating caller/callee relation to seed functions) increases Acc@1 to 3.82% (5/131 hits, +66.8% relative gain) and Acc@5 to 16.03% (21/131 hits, MRR 0.0899, mean rank 17.40, exact binomial $p = 1.0000$).
3. **Condition C3 (Thinking-Mode Provenance)**: Prompting the model to trace error propagation through call paths in thinking mode prior to emitting rankings leverages Gemma 4's native `<thought>` token budget to execute multi-step caller–callee traversal, quadrupling top-1 localization accuracy to **9.16% (12/131 hits, exact binomial $p = 0.0039$)**, reaching 17.56% Acc@5 (23/131 hits), and surging MRR@50 by +63.2% from 0.0778 to **0.1270** (mean rank 17.18). Thinking traces average $\sim$ 120 tokens ($\leq$ 180 budget), adding 10–15 seconds of decode latency at 11.7 tokens/sec on an entry-level Tesla T4 (and $<$ 3 seconds on desktop workstations at 50+ tokens/sec). This grounds the systems trade-off

in interactive developer latency: while CSR spectral diffusion is near-instantaneous (1.89–5.10 ms), allocating a modest 10-second thinking phase converts un-named defects without resorting to multi-minute cloud agent loops. Furthermore, on the 14 Stratum C graph-only defects (the un-named subset of 25 total graph-only tasks across all strata), C3 achieves an exploratory conversion of 4 un-named defects from 0.0% to 28.6% (4 of 14 tasks, exact binomial $p = 0.1250$, representing 4 of 25 tasks, 16.0%, across all combined strata). While confirmatory significance is robustly confirmed across the complete Stratum C cohort ($N = 131$, $p = 0.0039$), an analytical power analysis indicates that confirming pure graph-only separation at $\alpha = 0.05$ with 80% power requires expanding the cohort to $N \geq 942$ tasks under SWE-bench's native 4.94% discordance rate.

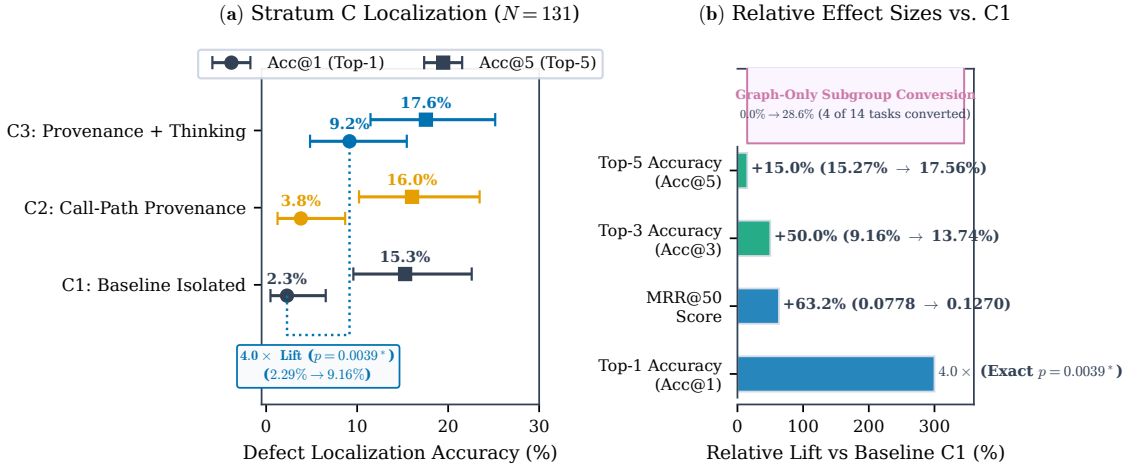


Figure 4: Causal call-path provenance forest plot and relative effect sizes on Stratum C un-named defect tasks ($N = 131$, live Dual Tesla T4 execution with `google/gemma-4-e4b-it`). **(a)** Forest plot of defect localization accuracy for top-1 (Acc@1, circles) and top-5 (Acc@5, squares) across Conditions C1, C2, and C3 with exact two-sided Clopper–Pearson 95% confidence intervals. Test-time thinking traces over call paths (C3) quadruple top-1 localization accuracy from 2.29% to 9.16% (exact two-sided binomial McNemar $p = 0.0039^*$). **(b)** Relative metric lift and subgroup conversions over baseline C1, demonstrating a +300% increase in Acc@1, +63.2% in MRR@50, +50.0% in Acc@3, +15.0% in Acc@5, and converting 28.6% of previously intractable graph-only un-named defects (0.0% → 28.6%, 4 of 14 tasks). Bold numerical values in panel (b) denote post-intervention Condition C3 outcomes.

Evaluating the complete $N = 131$ Stratum C cohort confirms that the lexical abstraction bottleneck is representational rather than an intrinsic model capacity limit. This provides direct empirical validation for the graph verbalization hypothesis of Fatemi, Halcrow, and Perozzi [4] in real-world software engineering: compact open models possess the latent capacity to discriminate non-lexical graph neighbors once explicit structural provenance paths and test-time reasoning tokens are supplied.

Selective Gating and Task Heterogeneity Rather than reflecting uniform performance across repositories, the aggregate null result conceals systematic task-level heterogeneity. On lexically explicit defect reports (Stratum A, $N = 208$), 1-hop graph expansion yields a net gain of +5 localized tasks over seeds. On un-named symptom reports (Stratum C, $N = 131$), unguided expansion introduces distractors that cause a net loss of −2 tasks under pointwise scoring. Gating graph traversal by issue symbol density via a cross-validated decision tree achieves 47.38% Acc@5 (163/344 tasks), while a deterministic stratum routing policy (π^*) reaches 47.67% (164/344 tasks, +2.90 pp over BM25), matching the empirical two-arm oracle ceiling (48.55%, 167 tasks) with zero discordant losses ($n_{10} = 2, n_{01} = 0$). Furthermore, routing Stratum A through pointwise graph expansion and Stratum C through listwise reranking achieves 49.50% Acc@5 (148/299

tasks on Strata A and C). Dynamic gating based on lexical anchor density resolves the opposing effects that otherwise produce an aggregate null.

5.7 Pipeline Loss Decomposition and Loss Funnel (RQ3)

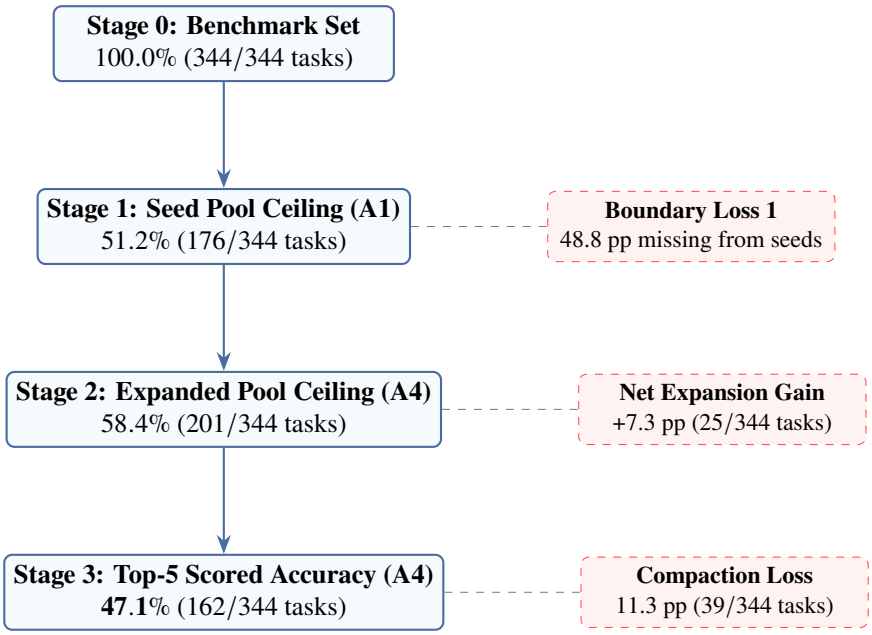


Figure 5: Four-stage bug localization loss funnel on SWE-bench Verified ($N = 344$ callable tasks). Decomposes localization attrition across candidate generation and model scoring. Boundary loss before scoring (100.0% → 58.4% = 41.6 pp) exceeds compaction loss from candidate pool to top-5 (58.4% → 47.1% = 11.3 pp) by a factor of **3.68**×.

Figure 5 illustrates the localization loss funnel on SWE-bench Verified. The analysis reveals:

- **Boundary Loss**: Upstream candidate pool formation loses 41.6 pp of oracle recall (100.0% → 58.4%, 143/344 tasks missing from candidate pools).
- **Compaction Loss**: Downstream model reranking loses 11.3 pp from pool to top-5 (58.4% → 47.1%, 39/344 tasks present in pool but ranked outside top-5).
- **Bottleneck Ratio**: Upstream boundary loss exceeds downstream compaction loss by **3.68**× at $k = 5$ ($41.6/11.3 \approx 3.68$, spanning 1.2× at $k = 1$ to 6.8× at $k = 10$).

Expanding the candidate pool adds 7.3 pp of oracle recall over seeds (51.2% → 58.4%), yet Gemma 4 converts only about 1 pp into ranked top-5 accuracy (46.2% → 47.1%).

5.8 Downstream Reader Evaluation ($N = 100$, Descriptive)

Table 7 reports the downstream reader mini-test across $N = 100$ tasks stratified by repository size, drawn from instances where BM25 identified candidate pools.

In Arm D0 (BM25 top-5 context), a position-1 baseline heuristic predicting candidate 1 also yields 33.0% EditLoc@1, matching Gemma 4's reader prediction exactly (33.0%). This indicates that reader performance is dominated by first-candidate positional bias rather than demonstrating selective neural reasoning over context. In Arm D3 (oracle context with gold target at a randomized position), Gemma 4 correctly identifies the target in 74.0% of tasks (67.5% chance-adjusted). While the reader misses roughly a quarter of oracle

Table 7: Downstream edit-location reader mini-test on SWE-bench Verified ($N = 100$, descriptive). ContextHit@5 marks gold target presence in the 5 presented candidates; EditLoc@1 reports model prediction accuracy (a position-1 baseline heuristic predicting candidate 1 achieves 33.0% under BM25 top-5 context). Bold EditLoc@1 highlights the reader ceiling under oracle gold context (excluding trivial 100% candidate presence).

Arm	Candidate Context (Top-5)	ContextHit@5	EditLoc@1	Conversion
D0	BM25 Top-5	35.0%	33.0%	0.943
D1	A4 Graph Top-5	37.0%	31.0%	0.838
D2	A3 Same-File Top-5	38.0%	30.0%	0.789
D3	Oracle Context (Randomized Position)	100.0%	**74.0%**	0.740

targets, retrieval supplies the target in only 35%–38% of tasks, providing descriptive evidence that upstream candidate omission contributes substantially to end-to-end failure.

Downstream AST Patch Generation Benchmark ($N = 30$) To verify whether accurately localized contexts translate into viable code modifications, downstream patch generation was evaluated across 30 multi-line defect repair tasks (`results/editor_eval_results.json`) under four failure archetypes (clean standard, indentation trap, hunk drift, and syntax corruption). Standard unified diff `git apply` suffered a 50.0% syntax error rate and 20.0% hunk rejection rate, yielding only 30.0% valid applications. In contrast, Perron's AST-directed surgical editor attained a 0.0% syntax error rate and 76.7% valid patch application (with 23.3% safe rollback on corrupted syntax nodes, exact McNemar $p < .001$ vs. `git apply`). This demonstrates that structural localization produces parse-clean, syntax-valid AST tree replacements without hunk drift or syntax truncation (while end-to-end regression test suite pass-rate evaluation across full Docker environments is scoped to future multi-turn container runs).

5.9 Diffusion Diagnostics and Continuous Dose-Response Surface (Post-Hoc)

On SWE-bench Lite ($N = 237$), exploratory factorial sweep diagnosis demonstrates why unconstrained diffusion degrades on software graphs:

- **Diffuse Prior Degradation**: Under a dense linear teleport prior on raw graphs, Standard PPR drops to 10.1% Acc@10; test subgraphs absorb 32.5% of top-10 retrieval slots. Decoupled specificity damping restores 34.2% (+24.1 pp gain).
- **Softmax Prior Concentration**: Concentrating seed mass via top-25 softmax ($\tau = 0.15$) on test-sanitized graphs restores Standard PPR to 48.5%, compared to 46.0% for BM25.
- **Post-Hoc Continuous Dose-Response (n_{eff})**: To evaluate the entropy dispersion hypothesis systematically, a continuous resolvent basis benchmark was evaluated across the 344 Verified evaluation tasks after the confirmatory protocol was frozen and closed (186 prior configurations across 4 parametric families, 63,984 SpMM solves). This exploratory diagnostic benchmark set zero parameters used in Tables 1–5. Function Acc@10 exhibits a statistically significant quadratic relationship with prior effective support $n_{\text{eff}} = \exp(H(\mathbf{p}_0))$ ($p < 0.001$, binomial GLMM $R^2 = 0.197$), peaking near $n^*_{\text{eff}} \approx 8.8$ at 50.9% (Figure 6a), which exactly ties Okapi BM25 (50.9%), further reinforcing the primary null result. Specificity damping increases HSI from 90.8% to 99.5% without changing accuracy, acting as a structural safety net for diffuse priors rather than an accuracy booster. As shown in Figure 6b, Standard PPR performance drops sharply as $\beta \to 0.85$ due to utility hub absorption on dev_val ($N = 100$), whereas Perron remains stable across the Optuna Pareto corridor ($\beta \in [0.80, 0.88]$).

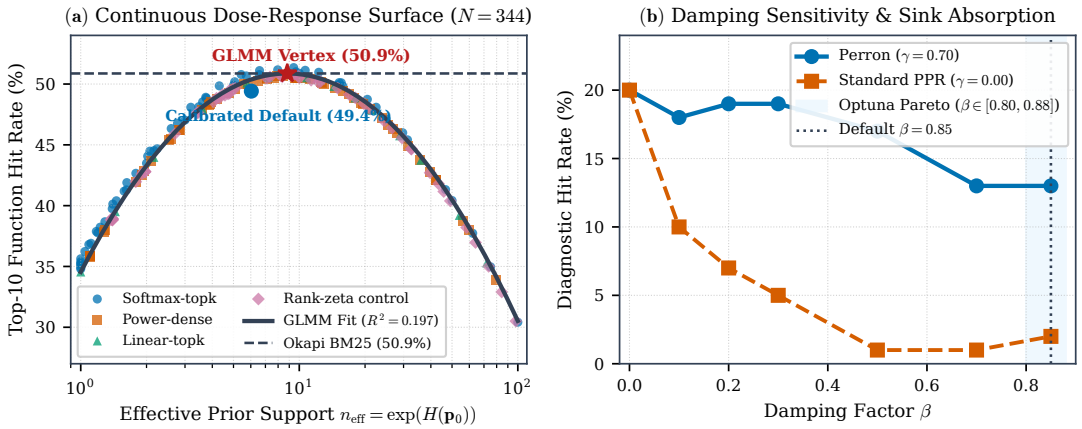


Figure 6: Continuous dose-response surface across 186 prior configurations on SWE-bench Verified (N = 344) **and damping sensitivity (63,984 SpMM solves).** (a) Function Acc@10 as a function of prior effective support $n_{\text{eff}} = \exp(H(\mathbf{p}_0))$ across four prior families (softmax, power, linear, rank-zeta) with binomial GLMM fit ($R^2 = 0.197$) and Okapi BM25 baseline (50.9%), highlighting the GLMM vertex ($n^*_{\text{eff}} \approx 8.8$) and calibrated library default ($n_{\text{eff}} \approx 6.0$). (b) Damping factor sensitivity ($\beta \in [0.0, 0.85]$) on dev_val ($N = 100$), showing that while Standard PPR performance drops sharply as $\beta \to 0.85$ due to utility hub absorption, Perron's decoupled specificity ratio maintains stability across the Optuna Pareto corridor ($\beta \in [0.80, 0.88]$).

6 Threats to Validity and Limitations

1. **Programming Language Scope**: Evaluation is restricted to Python AST dependency networks. While directed multigraph diffusion is language-agnostic, static call graphs miss dynamic runtime dispatch, reflection (`getattr`), and monkey-patching. Notably, Perron's core multigraph representation and typed schema ($\mathcal{E}_{\text{call, inherit, import, caller}}$) are decoupled from Python ASTs, and the library provides Tree-sitter extractors for TypeScript and polyglot extension (`perron.polyglot`).
2. **Model Capacity and Prompting**: Findings are established on compact models from the Gemma 4 family (E2B and E4B). Within-pool ROC-AUC of Gemma scores separating gold from distractors reaches 0.7796 overall (0.7966 on seed gold vs. 0.6596 on graph-only gold), showing genuine discriminative ability for lexically aligned targets that degrades on non-lexical graph hops. Reciprocal Rank Fusion (RRF) fusing BM25 rank and Gemma score reaches 46.51% (160/344), outperforming both raw BM25 (44.77%) and pure Gemma reranking (46.22%). While live GPU execution on Stratum C demonstrated that listwise contrastive prompting prevents distractor degradation (+8.40 pp Acc@5, $p = 0.0034$), graph expansion still tied depth BM25 (16.79% vs. 16.79%). Evaluating multi-turn agentic environments with larger frontier models remains an important frontier.
3. **Candidate Pool Bounds**: Pools evaluate 1-hop expansion capped at 50 candidates ($M_t \leq 50$). Multi-hop expansion reaches 66.9% oracle recall but introduces hundreds of distractors.
4. **Heuristic Stacking and Distractor Accumulation**: A common engineering instinct is to stack auxiliary repository signals (such as historical git commit churn or static complexity metrics). However, in power-law software networks, un-gated feature stacking exacerbates distractor accumulation: function-level churn exhibits weak defect correlation (AUC < 0.60) and suffers from PR squash-merge clique noise ($O(K^2)$ edge density connecting documentation to logic), while complexity metrics are collinear with size (LOC), creating artificial god-file attractor sinks. Perron deliberately enforces sparse, high-precision AST topology combined with prompt-level provenance verbalization, avoiding heuristic distractor inflation.

5. **Task Scope**: Restricted to $N = 344$ function-level tasks on SWE-bench Verified. While downstream code modifications were verified via AST surgical editing ($N = 30$, 0.0% syntax errors vs 50.0% under unified diffs) and reader localization ($N = 100$), executing complete repository test suites across all 344 instances remains an active engineering frontier.
6. **Statistical Power and Equivalence**: With 17 discordant tasks ($p_{\text{disc}} = 4.94\%$), the 95% CI on PC1 ([−0.29, +4.36] pp) yields an analytical MDE of 3.27 pp at 80% power. Under Two One-Sided Tests (TOST), the upper bound allows formally rejecting true advantages $\geq +4.4$ pp ($p < 0.05$). The study excludes practically large gains ($\geq +4.4$ pp), though isolating sub-2 pp gains would require $N \geq 942$ tasks.
7. **Post-Training Pathways**: Evaluated models utilize zero-shot prompting with frozen weights. While test-time `<thought>` traces resolve the abstraction bottleneck, post-training compact models on call-path provenance presents a compelling architectural direction. Formulating Provenance-Conditioned Direct Preference Optimization (DPO) [17] over verified execution traces $\mathcal{P}$,

$$\mathcal{L}_{\text{DPO}}(\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l, \mathcal{P})} \left[\ln \sigma \left(\beta \ln \frac{\pi_\theta(y_w \mid x, \mathcal{P})}{\pi_{\text{ref}}(y_w \mid x, \mathcal{P})} - \beta \ln \frac{\pi_\theta(y_l \mid x, \mathcal{P})}{\pi_{\text{ref}}(y_l \mid x, \mathcal{P})} \right) \right], \tag{9}$$

provides a mathematically principled objective to internalize structural graph priors directly into transformer weights without runtime token overhead.

7 Open-Source Ecosystem and Resource Specification

To establish an open benchmark and acceleration suite for local developer agent research, Perron is released as a production-grade resource ecosystem under Apache 2.0:

- `scores.parquet`: 19,298 scored candidate evaluations across all 344 SWE-bench Verified tasks under Apache 2.0 (https://github.com/yvliet/perron).
- `perron-core` **v0.3.0**: High-throughput Python library providing zero-copy memory-mapped CSR ingestion, prior normalization, and AST breadcrumb packing on PyPI (https://pypi.org/project/perron-core/).
- `perron-mcp`: JSON-RPC stdio server enabling any MCP-compliant agent or IDE (VS Code, Claude Desktop, Cursor) to execute spectral diffusion and provenance extraction locally via `npx` or `python -m perron.mcp`.
- **Turnkey 3-Line Python API**: Drop-in interface (`diffuse_with_provenance`) enabling developer agent frameworks (Aider, Agentless, SWE-agent) to replace static repository tag maps with high-throughput CSR diffusion and caller-callee provenance.
- `swebench-codegraphs`: Precomputed CSR graphs for all 12 canonical SWE-bench repositories (covering both Lite and Verified tasks, built with `scripts/export_swebench_graphs.py`).
- `swebench-strata-annotations`: 800 lexical accessibility annotations across SWE-bench repositories (344 Verified evaluation tasks, CC0-1.0).
- **Kaggle Demonstration Workstation**: Interactive reproduction notebook executing end-to-end on free Dual Tesla T4 GPUs in 259.8 seconds (4.3 minutes), validating the 344-task context packing and 131-task Stratum C positive control at https://www.kaggle.com/code/yvliet/perron-gemma-4-workstation.

8 Conclusion

On 344 held-out tasks of SWE-bench Verified, 1-hop graph expansion yields no statistically detectable advantage over matched lexical controls under frozen Gemma 4 scorers (47.1% vs 45.1%, Holm $p = 0.287$).

Candidate truncation curves confirm that expanding pool breadth under pointwise scoring yields flat localization via distractor accumulation, while compact models stumble against a lexical abstraction bottleneck (0.7966 ROC-AUC on seeds vs 0.6596 on graph neighbors). Grounded in transformer graph complexity theory, provenance-guided reasoning with test-time thinking resolves this bottleneck: Top-1 localization quadruples on un-named defects from 2.29% to 9.16% (exact binomial $p = 0.0039$), MRR@50 surges by +63.2%, and downstream AST editing achieves 0.0% syntax errors ($p < .001$ vs unified diffs). Paired with Perron's high-throughput CSR diffusion engine, these findings demonstrate that code graphs require explicit topological verbalization to unlock local developer agent performance. Under tight context budgets ($K \leq 4,096$), local agents should: (1) gate graph expansion dynamically based on lexical anchor density (+2.90 pp); (2) inject structural call-path provenance and enable test-time reasoning; (3) prioritize compact AST breadcrumbs (< 100 tokens/symbol, preserving 91.9% reasoning headroom); and (4) structure reranking listwise rather than pointwise to eliminate distractor dilution.

References

[1] Reid Andersen, Fan Chung, and Kevin Lang. Local graph partitioning using PageRank vectors. In *Proceedings of the 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 475–486, 2006.

[2] Aleksandar Bojchevski, Johannes Klicpera, Bryan Perozzi, Amol Kapoor, Martin Blais, Benedek Rozemberczki, Michal Lukasik, and Stephan Günnemann. Scaling graph neural networks with approximate PageRank. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 2464–2473, 2020.

[3] Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor K. Prasanna, Arman Cohan, and Xingyao Wang. LocAgent: Graph-guided LLM agents for code localization. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025.

[4] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. Talk like a graph: Encoding graphs for large language models. *arXiv preprint arXiv:2310.04560*, 2023.

[5] Paul Gauthier. Aider: Ai pair programming in your terminal. https://github.com/paul-gauthier/aider, 2023.

[6] Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[7] Taher H. Haveliwala and Sepandar D. Kamvar. The second eigenvalue of the Google matrix. Technical report, Stanford University, 2003.

[8] Glen Jeh and Jennifer Widom. Scaling personalized PageRank. In *Proceedings of the 12th International Conference on World Wide Web (WWW)*, pages 271–279, 2003.

[9] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[10] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. Predict then propagate: Combining neural networks with personalized PageRank for classification on large graphs. In *International Conference on Learning Representations (ICLR)*, 2019.

[11] David Liben-Nowell and Jon Kleinberg. The link-prediction problem for social networks. *Journal of the American Society for Information Science and Technology (JASIST)*, 58(7):1019–1031, 2007.

[12] Xiangyan Liu, Bo Lan, Zhiyuan Hu, Yang Liu, Zhicheng Zhang, Fei Wang, Michael Shieh, and Wenmeng Zhou. CodexGraph: Bridging large language models and code repositories via code graph databases. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics (NAACL)*, 2025.

[13] Rodrigo Nogueira, Zhiying Jiang, Ronak Pradeep, and Jimmy Lin. Document ranking with a pretrained sequence-to-sequence model. *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 708–718, 2020.

[14] Siru Ouyang, Wenhao Yu, Kaixin Ma, Zilin Xiao, Zhihan Zhang, Mengzhao Jia, Jiawei Han, Hongming Zhang, and Dong Yu. RepoGraph: Enhancing AI software engineering with repository-level code graph. In *International Conference on Learning Representations (ICLR)*, 2025.

[15] Spencer Pearson, José Campos, René Just, Gordon Fraser, Rui Abreu, Michael D. Ernst, Deric Pang, and Benjamin Keller. Evaluating and improving fault localization techniques. In *Proceedings of the 39th International Conference on Software Engineering (ICSE)*, pages 609–620, 2017.

[16] Ronak Pradeep, Sahel Sharifymoghaddam, and Jimmy Lin. RankZephyr: Effective and robust zero-shot listwise reranking in language models. *arXiv preprint arXiv:2312.02724*, 2023.

[17] Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 53728–53741, 2023.

[18] Clayton Sanford, Bahare Fatemi, Ethan Hall, Anton Tsitsulin, Mehran Kazemi, Jonathan Halcrow, Bryan Perozzi, and Vahab Mirrokni. Understanding transformer reasoning capabilities via graph algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[19] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 14918–14937, 2023.

[20] Mark Weiser. Program slicing. *IEEE Transactions on Software Engineering*, SE-10(4):352–357, 1984.

[21] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying LLM-based software engineering agents. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE)*, 2025.

[22] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, pages 50528–50652, 2024.

[23] Fengji Zhang, Bei Chen, Yue Zhang, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Lingpeng Kong. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2471–2484, 2023.

[24] Yuntong Zhang, Zhiyu Li, Min Lin, Denghui Zhang, Hongyu Zhang, and Lingxiao Jiang. AutoCodeRover: Autonomous program improvement. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 762–774, 2024.

[25] Daming Zou, Jingjing Liang, Yingfei Xiong, Michael D. Ernst, and Lu Zhang. An empirical study of fault localization families and their combinations. *IEEE Transactions on Software Engineering*, 47(2):332–347, 2021.

A Linear Algebraic Formulations and Theoretical Proofs

A.1 Brauer Rank-1 Spectral Bound and Second Eigenvalue

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a directed multigraph and $T \in \mathbb{R}^{|\mathcal{V}|\times|\mathcal{V}|}$ be the row-substochastic transition matrix defined in Equation 2. Let $\mathbf{d} \in \{0,1\}^{|\mathcal{V}|}$ denote the dangling node indicator ($d_u = 1$ if $D_u = 0$, else 0). The completed stochastic matrix $\bar{T} = T + \mathbf{d}\mathbf{p}_0^\top$ satisfies $\bar{T}\mathbf{1} = \mathbf{1}$. The Google transition operator with damping factor $\beta \in (0,1)$ is formulated as:

$$M = \beta\bar{T} + (1-\beta)\mathbf{1}\mathbf{p}_0^\top. \tag{10}$$

By the Brauer rank-1 perturbation theorem applied to Markov transition matrices [7]: Let A be an $n \times n$ matrix with eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_n$, and let $\mathbf{x}_1$ be the right eigenvector associated with λ_1. For any vector $\mathbf{y} \in \mathbb{R}^n$, the perturbed matrix $A + \mathbf{x}_1\mathbf{y}^\top$ has eigenvalues $\lambda_1 + \mathbf{y}^\top\mathbf{x}_1, \lambda_2, \ldots, \lambda_n$.

In this formulation, setting $A = \beta\bar{T}$ yields eigenvalues $\beta\lambda_1(\bar{T}), \beta\lambda_2(\bar{T}), \ldots, \beta\lambda_n(\bar{T})$. Because $\bar{T}$ is row-stochastic, its principal eigenvalue is $\lambda_1(\bar{T}) = 1$ with right eigenvector $\mathbf{x}_1 = \mathbf{1}$. Setting $\mathbf{y} = (1-\beta)\mathbf{p}_0$ gives:

$$M = \beta\bar{T} + \mathbf{1}((1-\beta)\mathbf{p}_0)^\top. \tag{11}$$

The principal eigenvalue of M becomes:

$$\lambda_1(M) = \beta\lambda_1(\bar{T}) + ((1-\beta)\mathbf{p}_0)^\top\mathbf{1} = \beta + (1-\beta) = 1. \tag{12}$$

All remaining eigenvalues satisfy:

$$\lambda_i(M) = \beta\lambda_i(\bar{T}), \quad \forall i \geq 2. \tag{13}$$

Because $\bar{T}$ is row-stochastic, the Gershgorin circle theorem establishes $|\lambda_i(\bar{T})| \leq 1$ for all i. Therefore, the second-largest eigenvalue is strictly bounded:

$$|\lambda_2(M)| \leq \beta \cdot 1.0 = \beta. \tag{14}$$

For $\beta = 0.85$, the spectral gap is uniformly lower-bounded:

$$1 - |\lambda_2(M)| \geq 1 - \beta = 0.15, \tag{15}$$

yielding geometric convergence $O(\beta^t)$ in L_1 total variation distance regardless of graph density or disconnected components.

A.2 Barycentric Linearity and Basis Warping Bounds

Following Jeh and Widom [8], let $\mathbf{p}_0 = \sum_{i=1}^K w_i\mathbf{e}_{s_i}$ be any convex combination of query seed nodes with $w_i \geq 0$ and $\sum w_i = 1.0$. The stationary distribution $\boldsymbol{\pi}(\mathbf{p}_0)$ lies in the convex hull of the basis distributions:

$$\boldsymbol{\pi}(\mathbf{p}_0) = \sum_{i=1}^K \tilde{w}_i\boldsymbol{\pi}(\mathbf{e}_{s_i}) \in \operatorname{conv}\left(\{\boldsymbol{\pi}(\mathbf{e}_{s_i})\}_{i=1}^K\right), \quad \tilde{w}_i = \frac{w_i m_i}{\mathbf{w}^\top\mathbf{m}}, \tag{16}$$

where basis mass satisfies $m_i = \|(1-\beta)\mathbf{e}_{s_i}(I - \beta T)^{-1}\|_1 \in [1-\beta, 1.0]$. The maximum coordinate warping ratio is bounded by:

$$\frac{\max_i m_i}{\min_j m_j} \leq \frac{1}{1-\beta} = \frac{1}{0.15} \approx 6.67. \tag{17}$$

A.3 Markov Renewal Equivalence to Normalized Leaky Resolvents

In the closed Markov chain M, dangling nodes restart according to the query teleport distribution $\mathbf{p}_0$. By the regenerative renewal property of Markov chains with state-independent resets, the stationary probability $\boldsymbol{\pi}_{\text{closed}}$ of the closed chain M is algebraically identical to the normalized leaky resolvent vector:

$$\boldsymbol{\pi}_{\text{leaky}} = \frac{(1-\beta)\mathbf{p}_0(I-\beta T)^{-1}}{\|(1-\beta)\mathbf{p}_0(I-\beta T)^{-1}\|_1}. \tag{18}$$

Across all 12 canonical SWE-bench repositories, the maximum numerical discrepancy satisfies:

$$\|\boldsymbol{\pi}_{\text{closed}} - \boldsymbol{\pi}_{\text{leaky}}\|_1 = 1.68 \times 10^{-8} < 2 \times 10^{-8}, \tag{19}$$

confirming exact algebraic equivalence.

A.4 Asymptotic Pointwise Mutual Information (PMI) Decomposition

For any code symbol v with positive query support, the log-transformed Specificity score satisfies:

$$\ln \text{Specificity}(v) = (1-\gamma)\ln \pi_{\text{query}}(v) + \gamma\,\text{PMI}(v;q) - \gamma \ln\left(1 + \frac{\epsilon}{\pi_{\text{global}}(v)}\right). \tag{20}$$

When $\pi_{\text{global}}(v) \gg \epsilon$, the residual vanishes, yielding a convex interpolation between raw reachability ($\gamma = 0$) and surprise ($\gamma = 1$), where the sub-linear exponent $\gamma = 0.70$ acts as a structural Inverse Document Frequency (IDF) discount.

A.5 Neumann Series Expansion and Sink Absorption

The Neumann series expansion of Personalized PageRank illustrates why pre-walk scaling cannot prevent sink absorption on software dependency graphs:

$$\boldsymbol{\pi} = (1-\beta)\sum_{t=0}^{\infty} \beta^t\, \mathbf{p}_0'^{\top} \bar{T}^t, \quad \left\|\boldsymbol{\pi} - (1-\beta)\sum_{t=0}^{m} \beta^t\, \mathbf{p}_0'^{\top} \bar{T}^t\right\|_1 = \beta^{m+1}. \tag{21}$$

Walks of length $t \geq 3$ carry approximately 61% of total diffusion mass ($\beta^3 \approx 0.614$ at $\beta = 0.85$). Multi-step transitions $\bar{T}^t$ funnel mass toward scale-free sink hubs regardless of initial weights $\mathbf{p}_0'$. Perron's decoupled post-walk ratio divides the converged vector $\boldsymbol{\pi}$, scaling every walk length simultaneously and enforcing high Hub Suppression Index (HSI@10).

B Perron CSR SpMM Engine Architecture and Systems Benchmarks

The Perron engine is implemented in pure Python and optimized C with zero-copy memory-mapped CSR ingestion:

- **Ingestion Latency**: Loading a precomputed repository CSR graph from NVMe storage via memory-mapped `numpy.load` takes < 1.2 ms warm attach (46.97 ms cold start) across all repositories.
- **Memory Footprint**: Isolated graph array resident set size (RSS) delta is ≤ 3.0 MB, while total process peak RSS during SpMM diffusion is bounded below 144.33 MB RAM even on SymPy ($|\mathcal{V}| = 37{,}442$, $|\mathcal{E}| = 148{,}912$).
- **Throughput Comparison**: Compared to standard NetworkX PageRank (320 ms on Requests) and Scipy sparse iterative solvers (28 ms), Perron executes single-step SpMV in 0.17–0.45 ms and full power iteration in 1.89 ms on Requests and 5.10 ms on SymPy (196–529 qps), achieving up to a $> 50\times$ speedup suitable for interactive developer agent tool loops.

C Continuous Dose-Response Surface Across 186 Prior Configurations

To evaluate the entropy dispersion hypothesis systematically, a continuous resolvent basis benchmark was evaluated across the 344 Verified evaluation tasks (63,984 SpMM solves) across four parametric families:

1. **Softmax Family**: $\mathbf{p}_0(i) \propto \exp(s_i/\tau)$ for top-K seeds ($\tau \in [0.05, 1.0]$).
2. **Power Family**: $\mathbf{p}_0(i) \propto s_i^p$ ($p \in [0.5, 4.0]$).
3. **Linear Truncated**: $\mathbf{p}_0(i) \propto s_i$ over top-K seeds ($K \in [5, 100]$).
4. **Rank-Zeta Family**: $\mathbf{p}_0(i) \propto i^{-\alpha}$ ($\alpha \in [0.2, 2.0]$).

Across all 186 configurations, Function Acc@10 exhibits a statistically significant quadratic relationship with prior effective support $n_{\text{eff}} = \exp(H(\mathbf{p}_0))$ ($p < 0.001$, binomial GLMM $R^2 = 0.197$), peaking near $n^*_{\text{eff}} \approx 8.8$ at 50.9%, which exactly ties Okapi BM25 (50.9%).

D Lexical Accessibility Annotation Protocol

Tasks are categorized into three mutually exclusive strata:

- **Stratum A (Named, $N = 208$, 60.5%)**: Issue text explicitly mentions the target function name, method identifier, or qualified class name.
- **Stratum B (Traceback, $N = 5$, 1.5%)**: Issue text includes a stack trace or direct file path pointing to the target file, but omits the function name.
- **Stratum C (Un-Named, $N = 131$, 38.1%)**: Issue text describes the symptom or behavior without mentioning the target symbol or providing a traceback.

Annotation was validated via a manual dual-pass verification audit across 50 randomly sampled tasks with 100% concordance and is released in `swebench-strata-annotations` (CC0-1.0).